# Advanced Spring MVC for High-Traffic Applications

## Q1. What are some advanced features or techniques in Spring MVC that are useful for high-traffic applications?

**A:** Two useful techniques are **asynchronous request processing** (using DeferredResult or Callable) to prevent thread exhaustion during slow I/O. Another technique is **Content Negotiation**, which automatically handles the correct serialization (JSON/XML) based on client request headers, keeping the code clean and efficient.

## Q2. How can caching be implemented in Spring MVC?

**A:** Caching is implemented using the **@Cacheable** annotation on service methods, which tells Spring to store the result of that method call in a cache (like Redis). For HTTP caching, you use the **ETag** and **Last-Modified** headers in your controller to instruct the client's browser or proxy server to use a locally cached version of the response.

## Q3. What are the strategies for asynchronous processing in Spring MVC?

**A:** The primary strategies are returning **DeferredResult** or **Callable** objects from the controller. This allows the primary thread (the servlet thread) to be released immediately back to the pool while the task runs in a separate worker thread. When the task finishes, the result is delivered back to the client via the original request context.

## Q4. How can you scale a Spring MVC application horizontally?

**A:** You scale horizontally by running multiple identical instances of the application behind a **load balancer**. To make this work, the application must be designed to be **stateless**, meaning no user data (session data) is stored locally on the server. All shared data must be managed externally in a central, distributed store like Redis.

## Q5. Explain how you can handle static resources at scale (CDN, cache headers, versioning).

**A:** To handle static resources at scale, you place them on a **Content Delivery Network (CDN)** to serve files closer to users. You ensure resources have long **Cache-Control headers** so browsers cache them for extended periods. You use **versioning** (e.g., appending a hash to the filename) so browsers pull a new file only when the content actually changes.

## Q6. How do you tune the DispatcherServlet for high throughput?

**A:** You tune the DispatcherServlet by configuring the **embedded servlet container's thread pool** size to match the available CPU cores and expected load. You should also ensure that the application uses non-blocking resources (like non-blocking I/O) as much as possible, preventing threads from being stuck waiting.

## Q7. How does Spring MVC behave differently in a microservices architecture vs monolith?

**A:** In a **monolith**, Spring MVC handles both the web layer and the view rendering. In a **microservices** architecture, Spring MVC is typically used only for the **REST API** layer. It acts as a controller, handling requests and calling business services, but it usually returns only data (JSON), delegating view rendering to a separate frontend.

## Q8. How do you integrate Spring MVC with enterprise systems (Kafka, Redis, external auth providers)?

**A:** Integration is achieved using **Dependency Injection (DI)**. Controllers call services, and services use Spring clients: **Spring Data Redis** for caching, **Spring Kafka** for messaging (queues), and **Spring Security** (configured for OAuth2/JWT) for external authentication. All these clients are injected as beans.

## Q9. How to avoid performance bottlenecks in view rendering and serialization?

**A:** Avoid bottlenecks by using efficient view templates (like **Thymeleaf**) and ensuring templates do not execute complex database queries. For **serialization** (JSON), ensure you use efficient DTOs (Data Transfer Objects) that contain only the necessary fields, minimizing the data transferred over the network.

## Q10. What is an optimal approach for handling file uploads at scale? (streaming upload, chunking)

**A:** The optimal approach is **streaming upload**. Instead of buffering the entire file into the server's heap memory, you stream the data directly to external persistent storage (like S3) using the InputStream. For very large files, **chunking** is used, where the client breaks the file into small, manageable pieces that are uploaded sequentially and reassembled on the server side.

## Q11. How do you integrate async request processing with WebAsyncTask / DeferredResult?

**A:** You integrate by having the controller method return a **DeferredResult** object. When the request hits the controller, the servlet thread is released immediately. The DeferredResult is handed to a worker thread which holds it until the long task finishes. Once complete, the worker thread sets the result onto the DeferredResult, which sends the response back to the client.

## Q12. How does Spring MVC compare to Spring WebFlux in high concurrency scenarios?

**A: Spring MVC** (blocking model) requires a larger thread pool because each request uses a thread for its full duration (thread-per-request). This limits concurrency. **Spring WebFlux** (non-blocking model) can handle much higher concurrency with a smaller number of threads because threads are not held while waiting for I/O operations to complete.

## Q13. What architectural patterns are needed to prevent thread exhaustion in MVC apps?

**A:** The main pattern is the **Asynchronous Request Processing** pattern (using DeferredResult or Callable) to prevent threads from being tied up waiting for slow I/O. Other patterns include using a **Circuit Breaker** (Resilience4J) on external dependencies, which immediately returns a fallback instead of waiting for a failing service.

## Q14. In high-traffic apps, how do you reduce session load and dependency? (stateless design, tokenization)

**A:** You reduce session load by transitioning to a **stateless design**, where no user-specific data is stored on the server side. Authentication is handled using **tokenization** (JWTs), which are self-contained and sent with every request. If session data is absolutely necessary, it should be externalized to a highly scalable store like **Redis**.